Comprehensive Professional & Technical Cheatsheet

Muhammad Faizan Yousaf | AI/ML Engineer | FAST-NUCES 2026

SECTION 1 — Professional Identity & Soft Skills

1.1 Core Value Proposition — Why Me?

Strength	What it means in practice
I build, not just study	Every AI concept I know is tied to a real project — RAG, fine-tuning, NLP, model evaluation. Not tutorials.
Full-stack AI mindset	I can take a model from training through deployment. I understand both the ML and the engineering side.
Fast learner with proof	Picked up PyTorch, Hugging Face, FAISS, and FastAPI across projects without formal training in any of them.
Honest about what I don't know	I say "I don't know, but here's how I'd find out" — not a confident wrong answer.
High work ethic	Completed two internships, four major AI projects, and a FYP while finishing 130+ credit hours of coursework.

How I Handle Weaknesses

- Weakness: Sometimes go deep into a problem before stepping back to check if the approach is right.
- What I do: Set a 20-minute timer. If stuck, I write out what I know, what I don't, and what I'd search next.
- Weakness: Less experience with large team codebases and PRs at scale.
- What I do: I actively read open-source code on GitHub and study how senior engineers structure PRs and code reviews.

1.2 Lessons from FAST-NUCES — What University Taught Me

Lesson	How it shaped my work ethic
Grades don't equal competence	Got a D in DS semester 3, rebuilt understanding through projects — now I can explain and implement every structure
Pressure reveals priorities	Finals + internship + FYP at once taught me to identify what's blocking progress vs. what's just noise.
Teaching yourself is faster	Best skills (PyTorch, RAG, Hugging Face) came from building projects, not sitting in lectures.
Consistency beats intensity	Slow steady work on FYP over 2 semesters beat classmates who crammed at the end.
Ask the right questions	Learned to ask "what problem does this solve?" before writing a single line of code.

1.3 Conflict & People Management

When you disagree with a teammate or senior:

- Step 1: Listen fully before responding — understand their reasoning, not just their conclusion.
- Step 2: Ask a clarifying question ('Can you help me understand why you chose this approach?')
- Step 3: Share your perspective with data or a specific example — not a feeling.
- Step 4: If unresolved, propose a small test or experiment to let the outcome decide.
- Step 5: If escalation is needed, document the disagreement factually and involve a neutral third party.

Navigating a toxic or difficult environment:

Situation	Response
Senior takes credit for your work	Document your contributions in writing (commits, emails, Slack). Bring it up privately first.
Colleague is consistently negative	Minimize unnecessary interaction. Stay professional. Don't mirror the behavior.
You're asked to do someone else's job	Clarify your role scope with your manager in writing. Say yes once, flag it the second time.
Team has no clear communication	Propose a weekly sync. Send written summaries after verbal discussions.
Manager micromanages	Proactively over-communicate status updates so they don't feel the need to ask.

1.4 Ethics & Compliance — Knowing the Line

Scenario	Ethical?	Legal?	What to do
Manager asks you to hide a bug from a client	No	Depends	Refuse. Document the request. Escalate if needed.
Using open-source code without attribution	No	Depends on license	Always check license. MIT = ok. GPL = must credit.
Training a model on user data without consent	No	No	Refuse immediately. This violates GDPR/PDPA.

Logging user behavior for product improvement	Sometimes	If disclosed in ToS	Ensure it's in the privacy policy. Anonymize data.
Being asked to overfit results for a demo	No	Grey area	Flag this clearly — misleading demos lead to bad products.
Working extra hours without pay	Grey area	Depends on contract	Know your employment contract. Track hours.

■ **If pressured to cross a line: Say 'I need to think about this and check if it aligns with company policy.' Never agree on the spot. Put your concern in writing via email.**

1.5 Growth Mindset & Stress Management

Continuous Learning System:

• Weekly: Read 1 ML paper or technical blog (Andrej Karpathy, Hugging Face blog, ArXiv).

• Monthly: Build or extend one project using a concept you just learned.

• Always: When you encounter something you can't explain, stop and understand it before moving on.

• Track: Keep a personal log of what you learned and what broke — this is your real learning record.

Under pressure — what actually works:

Pressure Type	Technique
Deadline pressure	Break into 2-hour blocks. Ship something imperfect over nothing. Perfect later.
Not understanding something	Write down exactly what you don't understand. Be specific. Then search or ask.
Interview nerves	Think out loud. They want to see your process, not just your answer.
Imposter syndrome	List 3 things you built from scratch. That's real. The feeling is not.
Overwhelm	Pick ONE task. Do it fully. Then pick the next. Don't context-switch under stress.

SECTION 2 — Technical AI/ML Mastery

2.1 ML Algorithms — Supervised, Unsupervised, Reinforcement

Algorithm	Type	When to use	Key metric
Linear Regression	Supervised	Continuous output prediction	RMSE, MAE, R^2
Logistic Regression	Supervised	Binary classification baseline	Accuracy, AUC-ROC
Decision Tree	Supervised	Interpretable classification/regression	Gini, Entropy
Random Forest	Supervised	Tabular data, handles noise well	Feature importance
XGBoost / LightGBM	Supervised	Best for structured/tabular data	RMSE, Log Loss
SVM	Supervised	High-dim data, small datasets	Margin, Kernel choice
K-Nearest Neighbors	Supervised	Simple baseline, small data	k value, distance metric
Naive Bayes	Supervised	Text classification, fast	Likelihood, Laplace smooth
K-Means	Unsupervised	Customer segmentation, grouping	Inertia, Silhouette score
DBSCAN	Unsupervised	Arbitrary shape clusters, outlier detect	Epsilon, MinPts
PCA	Unsupervised	Dimensionality reduction, visualization	Explained variance ratio
Autoencoders	Unsupervised	Anomaly detection, compression	Reconstruction loss
Q-Learning / PPO	Reinforcement	Game playing, robotics, optimization	Cumulative reward

Bias vs Variance — The Core Trade-off

Problem	Meaning	Signs	Fix
High Bias (underfitting)	Model too simple — misses patterns	High train + test error	More features, complex model, less regularization
High Variance (overfitting)	Model too complex — memorizes data	Low train error, high test error	More data, dropout, regularization, simpler model
Good fit	Balanced complexity	Low train + test error similar	You're done

2.2 NLP & Generative AI — Key Concepts

Concept	What it is	Your experience
Tokenization	Split text into tokens (words/subwords)	Used in Pseudo2Code, NMT projects
Embeddings	Dense vector representation of meaning	FAISS retrieval, RAG, NMT Bahdanau attention
Attention Mechanism	Model learns which parts of input to focus on	Bahdanau attention in BiLSTM NMT
Transformer	Self-attention based architecture — backbone of modern LLMs	Used BERT, GPT-2, T5 via Hugging Face
BERT	Encoder-only — good for classification, NER, QA	Used in NLP coursework
GPT-2 / GPT family	Decoder-only — generates text autoregressively	Fine-tuned GPT-2 for Pseudo2Code
RAG	Retrieve relevant docs + pass to LLM as context	Clinical Diagnostic RAG, AI Knowledge OS
Fine-tuning	Train pretrained model on your specific task/data	GPT-2 on SPOC dataset — val loss 0.65
Prompt Engineering	Craft inputs to guide LLM behavior without retraining	Clinical RAG system
Vector DB / FAISS	Store and search embeddings by semantic similarity	Digital Genei internship, RAG projects
BLEU Score	Measures n-gram overlap between output and reference text	NMT: BLEU 0.45+, Image Captioning: 0.1648

Current Trends in AI/ML (2024-2025)

Trend	What it means practically
LLM Agents	LLMs that take actions — search, code, call APIs. Built on ReAct / tool-calling patterns.
RAG over fine-tuning	For most enterprise tasks, RAG is cheaper and more accurate than full fine-tuning. Know both.
Multimodal models	GPT-4V, Gemini — text + image input. CV + NLP merging into one.
Quantization / LoRA	Running large models on small hardware. GGUF, QLoRA, 4-bit inference.

MLOps maturity	Model monitoring, drift detection, CI/CD for ML pipelines — becoming standard expectation.
Open-source LLMs	LLaMA 3, Mistral, Phi-3 — production-grade open models. Know at least one.

2.3 Problem-Solving Framework — Data to Deployment

Step	What you do	Tools
1. Understand the problem	Define input, output, success metric. Ask: classification or regression? Stakeholder questions?	Pen + paper, stakeholder interviews
2. Explore the data (EDA)	Check distributions, missing values, class imbalance, correlations	Pandas, Matplotlib, Seaborn
3. Preprocess	Handle nulls (impute/drop), encode categoricals, scale features, split train/val/test.	Scikit-learn, Pandas
4. Baseline model	Start simple. Logistic regression or XGBoost first. Measure. Then improve.	Scikit-learn
5. Feature engineering	Create new features from domain knowledge. Remove redundant ones.	Pandas, domain expertise
6. Model selection	Try 2-3 models. Use cross-validation. Compare on validation set.	Scikit-learn, PyTorch
7. Tune hyperparameters	Grid search or random search. Don't over-tune on validation.	Optuna, GridSearchCV
8. Evaluate on test set	Run once on held-out test set. Report final metrics honestly.	Scikit-learn metrics
9. Deploy	Wrap model in FastAPI or Flask. Containerize with Docker. Serve.	FastAPI, Docker, Streamlit
10. Monitor	Track prediction distribution over time. Detect drift. Retrain if needed.	Evidently, custom logging

2.4 Alternative Approaches — When Standard Models Fail

Problem	Try this instead
Model not learning (loss stuck)	Check learning rate (too high or low). Check for data leakage. Verify labels are correct.
Overfitting badly	Add dropout, L2 regularization, reduce model size, get more data, use data augmentation.
Not enough labeled data	Semi-supervised learning, data augmentation, transfer learning from pretrained model.
Class imbalance	SMOTE oversampling, class weights in loss function, threshold tuning on predictions.
LLM hallucinating	Switch from pure generation to RAG. Ground answers in retrieved documents.
Slow inference in production	Quantize model (INT8/INT4), use ONNX export, distill into smaller model, batch requests.
Tabular data with many NaN values	Tree models (XGBoost) handle NaN natively. Or use iterative imputation.
No GPU available	Use LoRA / QLoRA for fine-tuning. Use CPU-optimized models (distilBERT, ONNX).
Evaluation metric looks good but model fails in prod	Metric-task mismatch. Re-examine what metric actually reflects real user experience.

SECTION 3 — Common Interview Q&As; (Situational + Technical)

3.1 Behavioural / Situational

Q: Tell me about a time you failed.
➜ In Semester 3 I got a D in Data Structures. I underestimated how much the subject built on itself. I retook the concepts independently, applied them in projects, and now I can implement and explain every core structure. It taught me to front-load understanding before trying to code.

Q: How do you handle working under pressure?
➜ I break the work into the smallest possible unit that I can actually ship. Under pressure, I stop trying to do everything and focus on what unblocks others first. I also write down what I'm worried about — it's almost always a smaller problem than it felt.

Q: Describe a conflict with a teammate and how you resolved it.
➜ During FYP, my teammate and I disagreed on the ML model choice — he wanted a simpler model, I wanted XGBoost. Instead of arguing, I proposed we both prototype our choice over a weekend and compare results on the validation set. XGBoost won on F1. The data decided, not us.

Q: Why should we hire you over another fresh graduate?
➜ Most fresh graduates have coursework. I have projects with real metrics — BLEU 0.45 on an NMT system, perplexity 1.9 on a fine-tuned LLM, a deployed RAG system on real clinical data. I don't need to be taught what a vector database or attention mechanism is — I've built with them.

Q: Where do you see yourself in 2 years?
➜ Owning features end-to-end — from understanding the problem to deploying the solution. I want to be the person on the team who bridges ML research and software engineering. Not just calling APIs but understanding what's happening inside.

Q: What is your biggest weakness?
➜ I sometimes go too deep into one approach before stepping back. I've learned to timebox: 20-30 minutes on one path, then explicitly evaluate if I should pivot. It's a working system.

■ *Don't say 'I work too hard'. This answer is honest and shows self-management.*

Q: How do you stay current in AI/ML?
➜ I read the Hugging Face blog, Andrej Karpathy's content, and relevant ArXiv papers. More importantly, I try to implement what I read — that's when I find out what I actually understood versus what I just thought I understood.

3.2 Technical AI/ML Interview Questions

Q: What is the difference between overfitting and underfitting?
➜ Overfitting: model learns training data too well including noise — high train accuracy, poor test accuracy. Fix: more data, dropout, regularization. Underfitting: model too simple — misses the pattern even in training data. Fix: more features, complex model.

Q: What is RAG and when would you use it over fine-tuning?
➜ RAG (Retrieval-Augmented Generation) retrieves relevant documents at inference time and passes them as context to an LLM. Use RAG when your knowledge base changes frequently, when you need source attribution, or when fine-tuning is too expensive. Fine-tune when you need to change the model's style, behavior, or domain vocabulary permanently.

Q: Explain how attention mechanisms work.
➜ Attention lets the model learn which parts of the input to focus on when producing each output token. For each output position, it computes a weighted sum of all input positions — weights learned during training. Bahdanau (additive) attention was my implementation in the NMT project.

Q: What metrics would you use to evaluate a classification model?
➜ Accuracy (only useful when classes are balanced). Precision: of all predicted positives, how many are truly positive. Recall: of all actual positives, how many did we catch. F1: harmonic mean of precision and recall — use when class imbalance exists. AUC-ROC: overall discriminative ability of the model.

Q: How do you handle class imbalance?
➜ First check if it actually affects model performance — sometimes it doesn't. If it does: oversample minority class (SMOTE), undersample majority, use class_weight='balanced' in Scikit-learn, tune the classification threshold, or use a different metric like F1 or AUC-ROC.

Q: What is the difference between a transformer encoder and decoder?
➜ Encoder (BERT): reads the full input bidirectionally — good for understanding tasks (classification, NER, QA). Decoder (GPT): reads left-to-right only with masked attention — good for generation. Encoder-decoder (T5, BART): encode input, decode output — good for translation, summarization.

Q: Walk me through how you would build a text classification system.

➜ 1. Clean and tokenize text. 2. Convert to embeddings (TF-IDF baseline or BERT). 3. Train classifier (Logistic Regression baseline, then fine-tuned BERT). 4. Evaluate on held-out test set with F1 score. 5. Deploy via FastAPI endpoint. 6. Monitor prediction distribution over time.

Q: What is transfer learning and why does it matter?

➜ Starting from a model already trained on a large dataset (like BERT on Wikipedia) and adapting it to your specific task. It matters because training from scratch is expensive and requires huge data — transfer learning lets you get strong results with small task-specific datasets.

Know your projects cold. Every answer you give should connect back to something you actually built.